TTDS Pastpaper-Driven Speed Notes

Chinese explanations with English exam terms

Generated for TTDS revision

2026-05-21

Contents

使用方式

这份 speed note 是按 **pastpaper 考察风格** 反推出来的，不是按 lecture page-by-page 抄内容。TTDS 近三年考试的颗粒度很稳定：大量 `short answer`，少量但高分的 `calculation`，再加一些 applied/design 题。复习时优先看每节的 `Pastpaper signal` 和 `Exam template`。

关键术语保留 English，例如 `Precision`, `Rocchio`, `PageRank`, `Jelinek–Mercer smoothing`，因为考试是英文作答。中文只负责帮你理解直觉。

0. Pastpaper Style Map

Core thesis：TTDS 不太考长篇理论证明，更常考“给你一个具体 IR/text analytics 场景，你能不能写出定义、公式、步骤、优缺点和 failure cases”。

三年卷子实际考法

Year	Question	Topic style
2023 Q1	short answers	`Cranfield`, `smoothing`, `PageRank`, `relevance feedback`, `shingles`, `crawler`, `implicit feedback`
2023 Q2	calculation	`PageRank` with teleportation, two iterations
2023 Q3	discussion	`LLMs vs web search`, `freshness`, `attribution`, `RAG`
2023 Q4	calculation	`F1@8`, `R-precision`, custom `nDCG'`
2023 Q5	calculation	`expected mutual information`, contingency table
2023 Q6	short answers	`MI independence`, `pLSI vs LDA`, `Gibbs sampling`, `feature scaling`, `train/dev/test`
2023 Q7	applied explanation	`bias in sentiment classifier`, mitigation
2023 Q8	applied evaluation	imbalanced misinformation detection, choose `recall` or `F_beta`
2024 Q1	mixed short/calculation	preprocessing, crawling, `permuterm`, `v-byte + delta decoding`
2024 Q2	calculation	`P at recall`, `R@6`, `AP/MAP`, `F1@10`, `nDCG`

Year	Question	Topic style
2024 Q3	calculation	idf, Jaccard, LM smoothing, tf–idf ranking, Rocchio
2024 Q4	calculation	mutual information for word-class association
2024 Q5	short applied	manual vs automated content analysis
2024 Q6	short answers	unigram, mixture of unigrams, LDA, OOV, zero–shot classification
2024 Q7	short applied	query logs, click–through, learning to rank
2025 Q1	mixed short/calculation	preprocessing, LLM vs search, TFIDF log base, Heaps' law, nDCG, MRR, crawler security, RAG encoder
2025 Q2	design/calculation	probabilistic positional index for noisy ASR, tf/df/cf, precision-recall tradeoff
2025 Q3	calculation	mutual information
2025 Q4	short answers	latent in LDA, sentiment dictionary, topic labels, multi–class/multi–label, validation vs test
2025 Q5	calculation/explanation	PageRank without teleportation
2025 Q6	design	coding assistant RAG, vector DB, privacy

Exam grain

- Definition 題一般只要 definition + why it matters + one

example/caveat。

- Calculation 题一定要 show steps，最后三位小数。
- Applied 题要写 method + why suitable + limitation/failure case。
- 看到 ranked list，马上想到 P@k, Recall@k, F1, R-precision, AP/MAP, nDCG, MRR。
- 看到 word vs category table，马上想到 2x2 contingency table 和 mutual information。
- 看到 web graph，马上想到 PageRank mass flow、out-degree split、teleportation。

1. IR Basics and Relevance

Pastpaper signal: 2023 Q1(a) 直接考 `Cranfield paradigm`; 2023/2025 都有 `LLMs vs search engines`; 很多题默认你知道 `query`, `document`, `relevance`, `test collection`。

Core thesis: Information Retrieval 是在大规模 mostly unstructured document collection 中，按 user information need 找到 relevant documents。它不是 exact database lookup，而是 uncertainty management。

Must-know

- `Information need` 是用户真正想解决的问题；`query` 只是用户写出的短文本表达。
- `Relevance` 是 document 是否满足 information need，不只是 lexical overlap。
- `Effectiveness` 看检索结果是否相关；`efficiency` 看系统能否快速处理大规模 collection。
- `Bag-of-words` 忽略 word order 和 syntax，只保留 term occurrences/frequencies。好处是简单高效，坏处是丢失 phrase/negation/context。
- `IR architecture`: documents -> preprocessing -> indexing -> query processing -> scoring/ranking -> evaluation。

Cranfield paradigm

Reusable test collection 通常包括:

1. `document collection`
2. `topics / queries / information needs`
3. `relevance judgments / qrels`
4. `evaluation measure`

Pastpaper 2023 Q1(a) 只要前三项也能拿核心分，但写 evaluation measure 更完整。

Exam template

IR is the task of finding unstructured documents that satisfy a user's information need. A query is only an imperfect expression of that need, so relevance is uncertain and must be evaluated experimentally. In the Cranfield paradigm, a reusable test collection contains a document collection, topics/queries, relevance judgments, and evaluation measures, allowing systems to be compared under the same conditions.

Pitfalls

- 不要把 `query` 等同于 `information need`。
- 不要把 relevance 写成 "contains query words"。相关性可能来自 meaning、intent、freshness、authority。
- `accuracy` 通常不适合 IR，因为 true negatives 太多。

2. Text Laws: Zipf and Heaps

Pastpaper signal：2025 Q1(d) 考 Heaps' law 中 closed-domain vs general-domain 的 b; book exercises 也常让你画 rank–frequency 和 vocabulary growth。

Core thesis：Text collections 有稳定统计规律。Zipf's law 解释少数 high-frequency terms 和大量 rare terms；Heaps' law 解释 vocabulary size 随 collection size 增长。

Zipf's law

Rank terms by frequency. The frequency/probability of rank r term is roughly inversely proportional to r:

$$f(r) \approx \frac{c}{r}$$

or:

$$r \cdot f(r) \approx c$$

Log-log graph 中接近一条斜率约为 -1 的线。

Heaps' law

Vocabulary size V grows with token count N:

$$V = kN^b$$

Take logs:

$$\log V = \log k + b \log N$$

用 log-log 线性回归可估计 k 和 b。

Pastpaper answer for 2025 Q1(d)

Closed-domain collection expected to have smaller b, 因为 vocabulary growth slows sooner。General-domain collection keeps introducing new topics, names and terms, so vocabulary grows faster.

Exam template

Heaps' law states that vocabulary size grows as $V = kN^b$. The parameter b controls how quickly new vocabulary appears. A closed-domain collection should have a smaller b because documents reuse specialised vocabulary, whereas a general-domain collection keeps introducing new topics, entities and terminology.

Pitfalls

- N 是 token count，不是 document count，除非题目明确让你按 documents processing plot。
- 文档处理顺序会影响 early curve，但 large collection 下 overall estimate should stabilise。
- Zipf 的 c 不是 universal constant；不同 corpus、words/bigrams/combined curves 会不同。

3. Text Preprocessing

Pastpaper signal：2024 Q1(a) 考 `stemming` 和 `removing non-alphanumeric characters`; 2025 Q1(a) 考 `stopping` 和 `removing URLs`; 题型固定是 “explain step + advantage + drawback + example”。

Core thesis：Preprocessing 把 raw text 变成 index terms，目标是提升 matching 和 efficiency，但每一步都可能牺牲 meaning。

Pipeline

- `Tokenisation`: split text into tokens/terms.
- `Stopping`: remove very frequent function words.
- `Normalisation`: map surface variants to same form, e.g. case folding, punctuation handling.
- `Stemming`: reduce variants to stem, e.g. connected/connection -> connect.
- Query 和 document 必须用 compatible preprocessing，否则 indexed term 和 query term 对不上。

Stopping

Advantage: reduces index size and noise for bag-of-words retrieval.
Drawback: can remove important meaning in phrase/name queries.

Examples of problematic stopwords:

- `to be or not to be`
- `The Who`
- `The Office`
- `No Country for Old Men`
- `May election`
- `US policy`

Stemming

Advantage: improves recall by matching morphological variants.
Drawback: overstemming can reduce precision.

Example:

- connect / connected / connection -> connect may help.
- universe / university conflation would hurt.

Removing non-alphanumeric characters / URLs

Advantage: reduces sparse/noisy tokens.
Drawback: may delete important evidence:

- `C++`, `COVID-19`, `Ph.D.`, `example.com`, hashtags, emojis, source domains.

Exam template

Stemming reduces morphological variants to a common stem, improving recall because related forms can match. However, aggressive stemming can hurt precision through overstemming, where semantically different words are conflated. Stopword removal reduces index size and removes very common low-information words, but it can damage phrase queries, names and negation-sensitive queries.

Pitfalls

- 不要说 preprocessing 总是 improves retrieval。正确写法是 tradeoff between recall, precision, index size and meaning preservation。
- 删除 URL 在 tweet classification 中可能去噪，但 domain/source 本身可能是 credibility signal。

4. Indexing and Query Processing

Pastpaper signal：2025 Q2 考 non-deterministic `positional inverted index`; 2024 Q1(e) 考 compressed postings; 2024 Q1(c) 考 `permuterm index`。

Core thesis：Inverted index 是 search engine 的核心 lookup structure。普通 postings list 只回答 term 在哪些 docs 出现；positional index 还能回答 phrase/proximity。

Inverted index

Dictionary maps each term to a postings list:

$$t \to [(docID, tf, positions), ...]$$

Common stored information:

- `docID`
- `term frequency`
- `positions`
- `fields/extents`
- pointer to postings list

Phrase and proximity search

For phrase `"brown fox"`:

1. get postings for `brown` and `fox`
2. intersect docIDs
3. compare positions: `pos(fox)=pos(brown)+1`

For proximity `/k`, require positions within distance k.

2025 Q2: probabilistic positional index for ASR

At each position, store multiple candidate words with probabilities:

```
doc1:
position 1: creating:0.15, treating:0.4, ...
position 2: jobs:0.35, hobs:0.3, ...
```

Posting example:

```
jobs -> [(doc1, position=2, prob=0.35)]
creating -> [(doc1, position=1, prob=0.15)]
```

`tf` can be expected count:

$$tf(t, d) = \sum_{\text{positions } p} P(t \text{ at } p)$$

`df` can be number of documents where term appears as a candidate, or expected document count depending on design. State assumption clearly.

`cf` can be expected total occurrences across collection:

$$cf(t) = \sum_{d} \sum_{p} P(t \text{ at } p)$$

Precision/recall intuition

Compared with indexing only ASR top-1 term:

- recall increases because true word may appear among alternatives.
- precision may decrease because extra alternatives create false matches.
- probability-aware scoring can recover some precision.

Exam template

A positional inverted index stores, for each term, the documents and positions where it occurs. For noisy ASR, each position can contain a distribution over candidate words, so postings should include document id, position and probability. Term frequency can be treated as expected count by summing probabilities across positions. This improves recall over top-1 indexing, but may hurt precision unless probabilities are used in ranking.

5. Compression, Dictionaries and Wildcards

Pastpaper signal: 2024 Q1(c) directly asks `permuterm index`; 2024 Q1(e) asks `v-byte decoding + delta decoding` and original postings list.

Core thesis: Index compression reduces storage and I/O. Dictionary structures and wildcard indexes decide what kinds of term lookup are efficient.

Delta encoding

Store gaps rather than absolute docIDs:

```
postings: [18, 578067, 579224]
gaps:     [18, 578049, 1157]
```

Recover postings by cumulative sum.

Variable-byte encoding

Use one or more bytes per integer. Each byte contributes 7 data bits; one high bit marks continuation/termination depending on lecture convention.

Pastpaper 2024 uses convention: high bit 1 terminates a number, high bit `0` continues.

Permuterm index

For term `hello`, append `$`:

```
hello$, ello$h, llo$he, lo$hel, o$hell, $hello
```

Wildcard lookup:

- `*tion` -> `tion$*`, prefix lookup `tion$`
- `blo*k` -> `blo*k$`, rotate to `k$blo*`, prefix lookup `k$blo`

Main drawback: dictionary size blow-up. A term of length m creates $m + 1$ rotations.

Dictionary structures

- `hash dictionary`: average $O(1)$ lookup, poor for prefix/range search.
- `tree/B-tree`: supports prefix/range queries, slower $O(\log M)$, good for disk-scale dictionaries.

Exam template

Delta encoding stores gaps between increasing docIDs, which are usually smaller than absolute docIDs and therefore compress better. Variable-byte encoding then represents small gaps with fewer bytes. A permuterm index supports wildcard queries by storing all rotations of each term plus an end marker `$`; wildcard queries are rotated so

that $*$ appears at the end and can be answered by prefix lookup. Its main drawback is large dictionary expansion.

Pitfalls

- 先 v-byte decode 得到 gaps，再 cumulative sum 得到 postings。
- 不要把 df 和 postings length compression 混在一起; compression 是存储层，df 是统计量。

6. Ranked Retrieval: TF-IDF and Cosine

Pastpaper signal：2024 Q3(a,d) 考 `idf` 和 `tf-idf weighted sum ranking`; 2025 Q1(c) 考 log base 改变是否影响 ranking。Cosine 讲义重要，但近三年没有直接手算。

Core thesis：Ranked retrieval assigns a score to each document and returns top-k. TF-IDF rewards terms that are frequent in a document but rare in the collection.

Basic statistics

- $tf(t,d)$: term t count in document d
- $df(t)$: number of documents containing t
- $cf(t)$: total occurrences of t in the collection
- N: number of documents

IDF

$$idf(t) = \log_{10} \frac{N}{df(t)}$$

If $df(t) = N$, then $idf(t) = 0$.

TF-IDF term weight

Lecture-style:

$$w_{t,d} = (1 + \log_{10} tf(t,d)) \log_{10} \frac{N}{df(t)}$$

Pastpaper 2024 Q3(d) explicitly says:

```
Do not squash tf.
Do not normalise for document length.
```

So use simple weighted sum:

$$score(q,d) = \sum_{t \in q} tf(t,d) \cdot idf(t)$$

Cosine similarity

For non-normalized vectors:

$$\cos(\vec{q}, \vec{d}) = \frac{\vec{q} \cdot \vec{d}}{\|\vec{q}\| \|\vec{d}\|} = \frac{\sum_i q_i d_i}{\sqrt{\sum_i q_i^2} \sqrt{\sum_i d_i^2}}$$

For normalized vectors:

$$\cos(\vec{q}, \vec{d}) = \vec{q} \cdot \vec{d}$$

Length normalization prevents long documents from getting high scores just because they contain more words.

2025 Q1(c): changing log base

$$\log_2 x = \frac{\log_{10} x}{\log_{10} 2}$$

Changing log base multiplies log values by a positive constant. Under simple TF-IDF scoring, ranking usually unchanged. It may matter if mixed with other unscaled features or ties.

Exam template

TF-IDF gives high weight to terms that occur frequently in a document but appear in few documents overall. IDF is $\log(N/df)$, so terms appearing in every document get zero weight. A query-document score can be computed by summing TF-IDF weights for shared query terms, or by cosine similarity in a vector space model. Cosine uses normalized dot product, reducing unfair advantage for long documents.

Pitfalls

- Read the wording: if it says "do not normalise", do not use cosine.
- `df` counts documents, not occurrences.
- `cf` counts occurrences across the whole collection.

7. Probabilistic Retrieval and Language Models

Pastpaper signal: 2023 Q1(b) asks why `smoothing` matters; 2024 Q3(c) asks compute $P(fat|d3)$ with `Jelinek–Mercer smoothing`.

Core thesis: Language Model for IR ranks documents by how likely their document language model would generate the query. Smoothing is essential because unseen query terms otherwise give zero probability.

Probability Ranking Principle

Rank documents by estimated probability of relevance:

$$P(R = 1 \mid d, q)$$

If estimates are accurate, ranking by decreasing relevance probability is optimal for effectiveness.

Query likelihood model

Rank by:

$$P(q \mid M_d)$$

Under unigram independence:

$$P(q \mid M_d) = \prod_{t \in q} P(t \mid M_d)^{tf(t,q)}$$

MLE:

$$P(t \mid M_d) = \frac{tf(t,d)}{|d|}$$

Problem: if $tf(t,d) = 0$, then $P(t \mid M_d) = 0$, so whole query probability becomes zero.

Jelinek-Mercer smoothing

$$P(t \mid d) = \lambda P(t \mid M_d) + (1 - \lambda) P(t \mid M_c)$$

where M_c is the collection language model.

BM25 high-level

BM25 is a strong probabilistic ranking baseline. It combines:

- term frequency saturation

- inverse document frequency
- document length normalization
- query term matching

You usually do not need exact BM25 calculation unless formula is given, but know why it improves over raw TF-IDF.

Exam template

In the language modelling approach, a document is relevant if its language model is likely to generate the query. With a unigram document model, $P(q|M_d)$ is the product of query term probabilities. MLE estimates $P(t|M_d) = tf(t,d)/|d|$, but this gives zero probability to unseen query terms. Smoothing, such as Jelinek-Mercer smoothing, mixes the document model with the collection model to avoid zero probabilities and handle data sparsity.

Pitfalls

- λ close to 1 relies more on document; low λ relies more on collection.
- If a term is unseen in document, its smoothed probability is $(1-\lambda)P(t|M_c)$, not zero.
- Do not confuse language model `P(q|d)` with classifier `P(class|d)`.

8. IR Evaluation Metrics

Pastpaper signal: This is the highest-yield topic. 2023 Q4, 2024 Q2, 2025 Q1(e) all directly test ranked retrieval metrics.

Core thesis: IR evaluation rewards not only whether relevant documents are retrieved, but where they appear in the ranked list.

Set-based metrics

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 = \frac{2PR}{P + R}$$

More generally:

$$F_\beta = \frac{(1 + \beta^2)PR}{\beta^2 P + R}$$

$\beta > 1$ weights recall more.

Ranked metrics

`P@k`: precision among top k.
`R@k`: recall among top k.
`R-precision`: $P@R$, where R is number of relevant documents for that query.

Average Precision

For one query with R relevant documents:

$$AP = \frac{1}{R} \sum_{k=1}^{n} P@k \cdot rel(k)$$

Only ranks containing relevant documents contribute.

MAP

$$MAP = \frac{1}{|Q|} \sum_{q \in Q} AP(q)$$

For a single query, MAP equals that query's AP.

MRR

$$MRR = \frac{1}{rank\ of\ first\ relevant\ document}$$

For multiple queries, average reciprocal ranks.

DCG and nDCG

Common version:

$$DCG@k = \sum_{i=1}^{k} \frac{gain_i}{\log_2(i+1)}$$

Then:

$$nDCG@k = \frac{DCG@k}{IDCG@k}$$

where IDCG is DCG of ideal ranking.

Pastpaper tactics

- 2023 custom nDCG': follow given discount, do not force standard log discount.
- 2024 Q2 Precision at R=50%: if 8 relevant docs total, 50% recall means after retrieving 4 relevant docs; compute precision at the rank where 4th relevant appears.
- 2025 Q1(e): for graded relevance {2,1,3,0,3}, ideal top-5 is [3,3,3,3,3] if question says many relevance-3 docs exist.

Exam template

Precision measures how clean the retrieved set is, while recall measures how many relevant documents were found. For ranked retrieval, metrics must reward early relevant documents. AP averages precision at each relevant rank, MAP averages AP over queries, MRR focuses only on the first relevant document, and nDCG handles graded relevance by discounting lower-ranked gains and normalising by the ideal ranking.

Pitfalls

- In AP, divide by total relevant documents R, not number retrieved in top-k unless 题目明确。
- nDCG needs ideal ranking based on available/relevant gains stated in question.

- `F1@10` uses precision and recall computed only from top 10 retrieved docs.

9. Test Collections and Evaluation Campaigns

Pastpaper signal: 2023 Q1(a) asks Cranfield components; evaluation calculation questions assume you understand qrels/test collections.

Core thesis: IR is an experimental science. A reusable test collection lets systems be compared fairly, but constructing relevance judgments is expensive and imperfect.

Reusable test collection

- `document collection`
- `topics / information needs`
- `queries`
- `relevance judgments`
- `evaluation metrics`

Topic vs query

- `query`: short text submitted to system.
- `topic`: fuller description of information need, often with title, description and narrative.

Topics are better for assessors because they define relevance criteria.

Pooling

Because judging every document for every topic is impossible:

1. multiple systems submit top results
2. top results are pooled and deduplicated
3. assessors judge pool
4. unjudged documents often treated as non-relevant

Inter-annotator agreement

`Cohen's kappa` adjusts observed agreement by expected chance agreement:

$$\kappa = \frac{P(A) - P(E)}{1 - P(E)}$$

High agreement suggests judgments are more reliable.

Web search evaluation

Commercial systems often use:

- click logs
- query reformulation
- dwell time
- human side-by-side judgments
- A/B testing

- online metrics

Exam template

A Cranfield-style evaluation uses a fixed document collection, topics/queries, relevance judgments and metrics, so different systems can be compared under identical conditions. Since exhaustive judgments are impossible at web scale, evaluation campaigns often use pooling: judge the top documents returned by many systems and treat unjudged documents cautiously. Web search companies additionally use click logs, human preference judgments and A/B testing.

Pitfalls

- Clicks are implicit feedback, not clean relevance labels.
- Pooling may miss relevant documents retrieved only by new systems.
- High average metric does not reveal all important failure cases.

10. Query Expansion and Relevance Feedback

Pastpaper signal: 2023 Q1(d,g,i) asks explicit/implicit feedback and query similarity; 2024 Q3(e) asks `Rocchio`; 2024 Q7 asks query logs and click-through data.

Core thesis: Query expansion tries to bridge vocabulary mismatch. Feedback methods use user judgments, pseudo-relevant documents or logs to modify the query, but risk query drift.

Explicit vs implicit relevance feedback

- `explicit relevance feedback`: user directly marks documents relevant/non-relevant.
- Commercial web search rarely uses it because users do not want extra labeling burden.
- `implicit feedback`: clicks, dwell time, eye tracking, skips, reformulations.
- Implicit feedback is abundant but noisy and biased by rank/presentation.

Rocchio algorithm

Move query toward relevant centroid and away from non-relevant centroid:

$$\vec{q}_m = \alpha\vec{q}_0 + \beta\frac{1}{|D_r|}\sum_{\vec{d}_j\in D_r}\vec{d}_j - \gamma\frac{1}{|D_{nr}|}\sum_{\vec{d}_j\in D_{nr}}\vec{d}_j$$

Pastpaper 2024 Q3(e) sets $\alpha = \beta = \gamma = 1$, uses tf-idf vectors, and says do not normalize.

Query logs and click-through

Can improve search by:

- identifying similar queries that share clicked documents
- learning query expansion terms
- training `learning-to-rank` features
- detecting navigational intents and preferred URLs
- spelling correction/query suggestion

Query drift

Expansion can add wrong terms and shift query away from original intent, hurting precision.

Exam template

Relevance feedback improves retrieval by using evidence about which documents are relevant. Rocchio represents the query and documents as vectors, then moves the query toward the centroid of rele-

vant documents and away from non-relevant documents. Query logs and click-through data can provide implicit feedback for query expansion and learning-to-rank, but clicks are biased by rank and presentation, so they must be interpreted carefully.

Pitfalls

- `Eye tracking` can be implicit feedback, but gaze means attention, not necessarily relevance.
- Clicked result may be clicked because it was ranked high, not because it is best.
- If 题目给 exact α, β, γ，不要默认 lecture values.

11. Mutual Information and Corpus Comparison

Pastpaper signal: 2023 Q5, 2024 Q4, 2025 Q3 all test `expected mutual information` with contingency table. This is one of the most repeatable calculation topics.

Core thesis: Mutual information measures how informative a term's presence/absence is about class membership.

Variables

- $U = 1$: document contains term t
- $U = 0$: document does not contain term t
- $C = 1$: document belongs to target class
- $C = 0$: document does not belong to target class

2x2 table

	$C = 1$	$C = 0$	total
$U = 1$	N_{11}	N_{10}	$N_{1.}$
$U = 0$	N_{01}	N_{00}	$N_{0.}$
total	$N_{.1}$	$N_{.0}$	N

Careful with notation in exam formula: some papers may order N_{01} and N_{10} visually differently. The safest method is always define your own table clearly.

Formula

$$I(U;C) = \sum_{u \in \{0,1\}} \sum_{c \in \{0,1\}} P(u,c) \log_2 \frac{P(u,c)}{P(u)P(c)}$$

Using counts:

$$\frac{N_{11}}{N} \log_2 \frac{NN_{11}}{N_{1.}N_{.1}} + \frac{N_{10}}{N} \log_2 \frac{NN_{10}}{N_{1.}N_{.0}} + \frac{N_{01}}{N} \log_2 \frac{NN_{01}}{N_{0.}N_{.1}} + \frac{N_{00}}{N} \log_2 \frac{N}{N_{0}}$$

Interpretation

- If term and class are independent, $I(U;C) = 0$.
- High MI means term presence/absence helps identify class.
- A term can occur more often in a class but have lower MI if it also occurs widely outside the class.

Manual and automated content analysis

Manual content analysis:

1. define categories/themes
2. create codebook
3. sample and train annotators
4. annotate documents
5. measure agreement
6. resolve disagreements
7. analyse patterns

Automate when scale is large and labels are clear. Avoid automation when categories are nuanced, rare, high-stakes, or labels are unstable.

Exam template

Mutual information measures how much knowing whether a document contains a term reduces uncertainty about whether it belongs to a class. I would construct a 2x2 contingency table for term presence and class membership, then sum $P(u, c)$ log$(P(u, c)/(P(u)P(c)))$ over the four cells. If term and class are independent, MI is zero; higher MI indicates the term is more characteristic of the class.

Pitfalls

- Always include all four cells, not just N_{11}.
- Use total documents N across all classes.
- MI is not simply raw count in class.

12. Topic Models: Unigram, pLSI, LDA

Pastpaper signal: 2023 Q6(b,c), 2024 Q6(a,b), 2025 Q4(a,c) repeatedly ask pLSI vs LDA, Gibbs sampling, latent variables, topic labels, unigram vs mixture.

Core thesis: Topic models are probabilistic generative models that explain documents as mixtures of latent topics. They are useful for exploratory corpus analysis, not direct relevance ranking.

Unigram model

All words generated independently from one vocabulary distribution. It ignores documents/topics.

Mixture of unigrams

Each document chooses one latent topic, then all words in that document are generated from that topic. Better than unigram, but one topic per document is unrealistic.

pLSI

Models document-specific topic mixtures, but topic proportions are parameters tied to training documents. Main limitation: no proper generative model for unseen documents and number of parameters grows with documents.

LDA

Latent Dirichlet Allocation adds Dirichlet priors:

- each document has topic mixture θ_d
- each topic has word distribution ϕ_k
- each word has latent topic assignment z
- observed: words and documents
- latent: topics, topic assignments, topic mixtures

Why LDA generative process is unrealistic

It assumes bag-of-words and generates each word from a chosen topic independently, ignoring syntax, word order and discourse. Example: sentence grammar is not generated realistically.

Gibbs sampling intuition

To assign a new topic to a word, consider:

- how common topic k is in this document
- how likely topic k generates this word

Then sample topic according to proportional probabilities.

Topic labels

LDA does not generate human-readable topic labels. Labels are assigned by humans, often by inspecting top words/documents, or by an external labeling method.

Exam template

In LDA, the observed data are the words in documents, while the latent variables include topic assignments for words, document-topic mixtures and topic-word distributions. LDA assumes each document is a mixture of topics and each word is generated by first choosing a latent topic and then a word from that topic. Topic labels are not produced by the model; humans usually interpret topics from top words or representative documents.

Pitfalls

- Topics are latent distributions, not predefined classes.
- Topic model output is not the same as document relevance.
- Do not say LDA "understands" topics; it finds statistical co-occurrence patterns.

13. Web Search, Crawling and Duplicates

Pastpaper signal: 2023 Q1(e,f,h,j), 2024 Q1(b,d), 2025 Q1(f) all test web crawling, shingles, user needs, back queues, freshness/security.

Core thesis: Web search adds scale, freshness, spam, duplicate content, user intent and crawling constraints to ordinary IR.

Web search user needs

Classic categories:

- `informational`: find information
- `navigational`: reach a specific site/page
- `transactional`: do something, e.g. buy, download, book

Crawler workflow

1. start from seed URLs
2. fetch pages
3. parse links
4. normalize URLs
5. check robots/policies
6. add unseen URLs to frontier
7. schedule revisits for freshness
8. store pages for indexing

Politeness vs freshness

- `politeness`: do not overload hosts; obey robots/crawl delay.
- `freshness`: revisit pages often enough to keep index up to date.

Balance with per-host delays, back queues and adaptive revisit frequency.

URL frontier

- front queues: priority/freshness/importance
- back queues: host-based scheduling and politeness

2023 Q1(j) asks role of back queues: enforce waiting time between requests to same host.

Hacked websites / crawler mitigation

Problem: hacked content pollutes index, serves irrelevant/spam/malware content, damages trust and ranking quality.
Mitigation: recrawling/freshness mechanism, change detection, content comparison, malware/spam detection, host reputation.

Duplicate detection

`Shingle`: contiguous k-gram of words/characters from a document. Used for near-duplicate detection with Jaccard/MinHash.

SEO and spam

Early ranking signals can be gamed by keyword stuffing, hidden text, cloaking, link farms, anchor text spam. Modern systems use many signals and spam detection.

Exam template

A polite crawler spaces requests to the same host, follows robots rules and uses host-based back queues to avoid overloading servers. At the same time, it must maintain freshness by revisiting important or frequently changing pages. Shingles represent documents as sets of contiguous k-grams, enabling near-duplicate detection using overlap measures such as Jaccard or MinHash.

Pitfalls

- Freshness is not "crawl everything constantly" ; it is constrained by politeness and resources.
- Shingle is not a hash itself; it is the k-gram representation used before similarity/hashing.
- User intent categories are not relevance metrics.

14. PageRank and Link Analysis

Pastpaper signal: 2023 Q1(c), 2023 Q2, 2025 Q5. PageRank is a very likely calculation/explanation topic.

Core thesis: PageRank is query-independent link-based importance. It models a random surfer moving through the web graph.

Meaning

`PageRank(page x)` is the stationary probability that a random surfer is at page x at a random time.

Formula with teleportation

$$PR_{i+1}(x) = \frac{1-\lambda}{N} + \lambda \sum_{y \to x} \frac{PR_i(y)}{L_{out}(y)}$$

where:

- N: number of pages
- λ: probability of following links
- $1-\lambda$: teleportation probability
- $L_{out}(y)$: out-degree of page y

Calculation steps

1. initialize all pages equally, e.g. $1/N$
2. for each target page, list its inlinks
3. each inlink y contributes $PR(y)/outdegree(y)$
4. multiply link contribution by λ
5. add teleportation base $(1-\lambda)/N$
6. repeat for required iterations

Without teleportation

Use only link-following mass:

$$PR_{i+1}(x) = \sum_{y \to x} \frac{PR_i(y)}{L_{out}(y)}$$

Teleportation is usually needed because:

- avoids rank sinks and cycles trapping probability
- handles dangling/dead-end pages
- helps convergence to unique stationary distribution
- models users jumping to arbitrary pages

Anchor text

Clickable link text describing target page. It can act like human-provided query expansion and may be weighted by source page importance.

Exam template

PageRank represents the stationary probability of a random surfer being on a page. In each iteration, a page receives contributions from incoming links, where each linking page splits its PageRank equally among outgoing links. With teleportation, every page also receives $(1-\lambda)/N$, preventing dead ends and cycles from trapping probability mass. PageRank is query-independent importance and is usually combined with content relevance in ranking.

Pitfalls

- Do not give full PR of a page to every outlink; divide by out-degree.
- Teleportation factor wording may vary. In TTDS 2023, formula used $\lambda = 0.5$ as link-following/damping factor.
- PageRank is not textual relevance.

15. Text Classification, Data Splits and Bias

Pastpaper signal: 2023 Q7/Q8, 2023 Q6(d,e), 2024 Q6(c,d), 2025 Q4(b,d,e). This topic is usually short answer or applied evaluation.

Core thesis: Text classification maps text items to predefined labels. Exam answers should focus on representation, training/evaluation split, class type, imbalanced metrics and bias/failure cases.

Classification types

- `binary classification`: two classes.
- `multi-class classification`: exactly one class among more than two.
- `multi-label classification`: multiple labels may apply to same item.

Pipeline

1. collect labelled data
2. split into train/dev/test
3. preprocess/represent text
4. train model
5. tune hyperparameters on dev/validation set
6. final evaluation on test set

Train/dev/test

- `training set`: fit model parameters
- `development/validation set`: tune hyperparameters/model choices
- `test set`: final unbiased evaluation

Never tune on test set.

Feature scaling

Transform features onto comparable ranges so large-scale features do not dominate learning algorithms.

OOV and transfer learning

Traditional classifiers with fixed vocab suffer from `out-of-vocabulary words`. Approaches: unknown token, subword features, character n-grams, update vocabulary. Transfer learning/pretrained language models reduce issue using subword tokenization and contextual representations.

Zero-shot classification

Classify into labels without task-specific training examples, often using pretrained language model/instruction or label descriptions. Useful when labelled data is scarce.

Imbalanced classification

Accuracy may be misleading. Choose metric based on cost:

- misinformation/moderation: high recall or F_β with $\beta > 1$
- high false-positive cost: precision
- balanced performance: F1

Bias in sentiment classifier

Why issue occurs:

- biased training data: identity terms co-occur with negative contexts
- spurious correlations
- representation bias
- lack of counterfactual examples

Mitigation:

- augment counterfactual data by swapping identity terms
- balance dataset across groups
- evaluate subgroup performance
- debias features/representations
- human audit and fairness constraints

Sentiment dictionary vs ML

Use sentiment dictionary when dataset is small, interpretability matters, domain is simple, and no labelled data. ML is better for context, sarcasm, domain-specific meaning, but needs labelled data.

Exam template

Text classification assigns documents or sentences to predefined classes. A good workflow uses training data to fit the model, a validation/development set to tune hyperparameters, and a held-out test set for final evaluation. For imbalanced tasks such as misinformation detection, accuracy is misleading; recall or F_β may be more appropriate if missing harmful content is costly. Bias can arise from spurious correlations in training data, so counterfactual data augmentation and subgroup evaluation are important mitigations.

Pitfalls

- `validation test` in 2025 means dev/validation set, not final test.
- Multi-class means one label; multi-label means possibly several labels.
- High recall system may overwhelm moderators if precision is ignored.

16. LLMs, Web Search and RAG

Pastpaper signal: 2023 Q3 and 2025 Q1(b) ask whether LLMs replace web search; 2025 Q1(g) asks why updating document encoder in large RAG is impractical; 2025 Q6 asks coding-assistant RAG design and privacy.

Core thesis: LLMs complement search engines rather than simply replacing them. RAG combines parametric model knowledge with retrievable, updateable non-parametric memory.

LLMs vs web search

Search engines still provide:

- `freshness`: up-to-date web index
- `attribution`: URLs/snippets for verification
- `navigational search`: find a specific website
- `transactional search`: buy/book/download
- `coverage`: web-scale current content
- `diversity`: multiple sources and viewpoints

LLMs provide:

- synthesis
- conversational interaction
- explanation
- summarisation
- query reformulation

Best answer: LLMs change search interfaces; they do not make retrieval unnecessary.

RAG architecture

1. user query
2. retrieve relevant documents/chunks from external corpus/vector DB
3. put retrieved evidence into prompt/context
4. LLM generates grounded answer
5. optionally cite sources or verify answer

Parametric vs non-parametric memory

- `parametric memory`: knowledge stored in model weights.
- `non–parametric memory`: external documents/index/vector database retrieved at runtime.

Why not update document encoder in large RAG

Document embeddings are precomputed and stored. If the document encoder changes, existing embeddings become stale, so the whole

corpus must be re-embedded and re-indexed. This is expensive at scale. Updating query encoder/reranker may be more practical.

Coding assistant RAG

Vector DB should index project-specific documents:

- local source code files
- README and documentation
- API specs
- config files
- tests
- commit/history if available
- coding conventions

Privacy-preserving redesign:

- run model locally or inside organization infrastructure
- keep vector DB and embeddings local
- avoid sending code to third-party API

Drawback: higher compute cost, model quality/latency may be worse, maintenance burden.

Exam template

LLMs do not remove the need for web search because search provides freshness, attribution, navigational access, transactional support and broad coverage. A RAG system combines retrieval with generation: it retrieves relevant external documents as non-parametric memory and uses an LLM to generate an answer grounded in that evidence. In large-scale dense retrieval, updating the document encoder is impractical because all stored document embeddings must be recomputed and re-indexed.

Pitfalls

- Do not say RAG is only "put documents into prompt"; retrieval quality and chunking/indexing matter.
- For coding assistant, vector DB should use the user's local project files, not just generic training data.
- Privacy fix "encrypt API call" is not enough if code still leaves machine; stronger answer is local/self-hosted deployment.

17. Calculation Survival Kit

When you see ranked list with + / -

1. mark relevant ranks.
2. compute total R.
3. for P@k, count positives in first k.
4. for Recall@k, positives in first k divided by R.
5. for F1@k, compute $P@k$, $R@k$, then $2PR/(P+R)$.
6. for AP, compute precision only at relevant ranks, average over R.
7. for nDCG, compute actual DCG, ideal DCG, divide.

When you see word-class table

1. identify target word and target class.
2. build 2x2 table: word present/absent vs class yes/no.
3. compute row/column totals.
4. plug four cells into MI formula.
5. interpret: independence gives 0; raw frequency alone is not MI.

When you see PageRank graph

1. write initial values.
2. write out-degree for each node.
3. for each target node, list incoming nodes.
4. sum incoming $PR/outdegree$.
5. add teleportation if required.
6. rank nodes after each requested iteration.

When you see TF-IDF ranking

1. compute df for query terms.
2. compute $idf = \log(N/df)$.
3. read instruction: squash tf? normalize? use cosine?
4. score each doc only on query terms unless instructed otherwise.
5. show ties explicitly.

When you see Rocchio

1. build original query vector.
2. compute relevant centroid.
3. compute non-relevant centroid.
4. apply $\alpha q_0 + \beta centroid_r - \gamma centroid_{nr}$.
5. discuss added/removed terms and query drift.

When you see LM smoothing

1. compute document probability $tf(t,d)/|d|$.
2. compute collection probability $cf(t)/|C|$.
3. combine with $P(t|d) = \lambda P(t|M_d) + (1-\lambda)P(t|M_c)$.

4. if query has multiple terms, multiply probabilities or log-sum them.

18. What To Prioritize

Tier 1: almost guaranteed calculation/short-answer value

- `IR evaluation metrics`: P/R/F1, R-precision, AP/MAP, MRR, DCG/nDCG.
- `Mutual information`: 2x2 contingency table.
- `PageRank`: with and without teleportation.
- `Preprocessing`: advantage/drawback/example.
- `TF-IDF / IDF / LM smoothing / Rocchio`.

Tier 2: recurring short-answer value

- `Cranfield paradigm`
- `LDA / pLSI / Gibbs / latent variables`
- `content analysis`
- `classification splits and imbalanced metrics`
- `crawler politeness/freshness/back queues`
- `shingles and duplicate detection`
- `query logs/click-through feedback`

Tier 3: applied discussion/design

- `LLMs vs search engines`
- `RAG architecture`
- `privacy-preserving local RAG`
- `probabilistic index for noisy ASR`
- `bias in sentiment classification`

One-page mental model

TTDS exam asks whether you can move from raw text to useful decisions:

```
raw text
-> preprocessing
-> index / representation
-> retrieval or classification model
-> ranking / prediction
-> evaluation
-> error analysis and mitigation
```

If stuck in an exam, answer with this pattern:

1. define the method
2. write the formula or workflow
3. explain why it helps
4. give one limitation/failure case
5. connect to evaluation